# FastGainRouter: Input-Adaptive Efficient Restoration for Smartphone Images with Dirty Lenses

**Donggyun Seo**
Department of AI Software
Soongsil University
seo6347@soongsil.ac.kr

**Abstract.** Smartphone images captured through dirty lenses exhibit diverse degradations, including dust, fingerprints, water droplets, scratches, and mixed artifacts. The SIDL benchmark studies this real-world restoration problem under fixed-model inference, where the same restoration model is applied to all inputs. This work asks a complementary question: *does every dirty-lens image require the same restoration budget?* We propose **FastGainRouter**, a lightweight input-adaptive framework that chooses among three branches: *Input* for bypassing restoration, *Tiny* for low-cost restoration, and *Base* for stronger restoration. Rather than predicting unstable hard oracle labels, the router predicts restoration gains of Tiny over Input and Base over Tiny. On SIDL validation, the proposed router provides favorable PSNR-latency operating points: a balanced mode slightly improves PSNR over Always Base while reducing latency, and an efficient mode yields larger latency reduction with only a small PSNR loss. These results suggest that adaptive restoration budget selection is a useful complement to fixed-model dirty-lens restoration.

**Keywords:** dirty-lens restoration, SIDL, dynamic inference, efficient image restoration, routing.

## 1 Introduction

Smartphone cameras are widely used in daily life, but their image quality can be degraded by contaminants on the lens surface. Water droplets, dust, fingerprints, scratches, and mixed contaminants can introduce spatially non-uniform blur, light scattering, local occlusion, contrast reduction, and color distortion. These artifacts differ from standard synthetic degradations such as Gaussian noise or motion blur because the degradation is introduced before image formation and varies across spatial regions and scenes.

The SIDL benchmark introduced a real-world dataset for restoring smartphone images captured through dirty lenses [3]. It provides paired degraded and clean reference images and evaluates representative restoration models under a fixed-model setting. Its Easy, Medium, and Hard categories are used for evaluation and analysis, not as routing signals for selecting different restoration models. Thus, the benchmark primarily asks how well a given restoration model performs on dirty-lens images.

We study a complementary direction: *input-adaptive restoration budget selection*. In dirty-lens restoration, not all inputs require the same computation. Some images are already close to the reference and can be damaged by unnecessary restoration. Many images can be restored sufficiently by a small model. Only a subset requires a stronger model. This motivates a routing framework that decides whether to skip restoration, use a lightweight branch, or use a stronger branch for each input.

We introduce **FastGainRouter**, which selects among three branches: **Input**, which returns the degraded image unchanged; **Tiny**, a width-4 NAFNet-based restoration model; and **Base**, a width-8 NAFNet-based model. Instead of directly classifying the best branch, FastGainRouter predicts two gains: the expected improvement of Tiny over Input and the additional improvement of Base over Tiny. This gain-based formulation avoids treating very small metric differences as hard class boundaries and better matches the quality-efficiency trade-off.

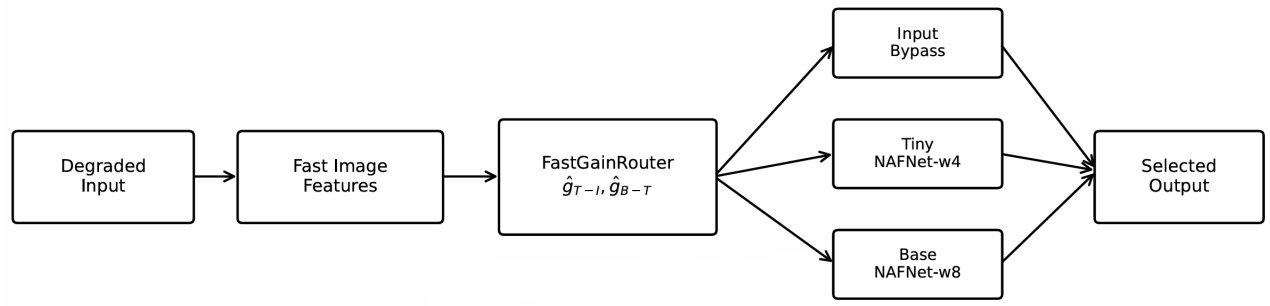



**Figure 1:** Overview of FastGainRouter. The router computes lightweight image statistics from the degraded input, predicts restoration gains, and selects Input, Tiny, or Base. Ground truth, difficulty labels, and metadata are not used at test time.

Our contributions are:

- We identify that over-restoration occurs in SIDL: Input, Tiny, and Base are each optimal for a non-trivial subset of validation samples.
- We formulate adaptive restoration as gain prediction rather than hard branch classification, using lightweight image statistics at test time.
- We show that FastGainRouter can reduce average component-based latency while maintaining comparable restoration quality to fixed Base inference.

## 2 Related Work

**Dirty-lens image restoration.** Image restoration has traditionally focused on denoising, deblurring, super-resolution, deraining, and dehazing. Dirty-lens restoration differs because contaminants lie in front of the camera lens and create spatially varying scattering and occlusion. SIDL addresses this problem by collecting paired smartphone images with realistic lens contaminants [3]. Our work does not replace the SIDL benchmark; rather, it explores whether restoration computation should be allocated adaptively across inputs.

**Deep image restoration.** Modern restoration backbones include multi-stage CNNs, transformers, and simplified convolutional baselines. MPRNet uses a multi-stage progressive design [8], Restormer introduces an efficient transformer for high-resolution restoration [7], and NAFNet provides a simple activation-free restoration baseline [2]. We use NAFNet variants only to construct controlled restoration branches. The focus of this work is not a new backbone, but when to invoke each branch.

**Dynamic inference.** Dynamic inference methods allocate computation according to input difficulty. BranchyNet uses early exits [4], SkipNet learns dynamic layer skipping [5], Slimmable Networks vary network width [6], and Once-for-All trains a super-network for many deployment constraints [1]. Unlike classification-oriented dynamic inference, our router predicts restoration gains for dense image restoration outputs.

## 3 Problem Setup and Method

Given a degraded dirty-lens input image $x$, a fixed restoration pipeline applies one model to all inputs. We instead define a branch set

$$\mathcal{A} = \{\text{Input}, \text{Tiny}, \text{Base}\},$$

and select a branch $a(x) \in \mathcal{A}$ for each sample. The Input branch is the identity path, Tiny is a lightweight restoration model, and Base is a stronger restoration model.

### 3.1 Branch Design

The **Input** branch is included because restoration is not always beneficial. Some validation samples are closer to the reference before restoration, so forcing them through a restoration model may introduce smoothing, color shifts, or texture changes.

The **Tiny** branch is a width-4 NAFNet-based model. It serves as a low-cost restoration path for inputs where a small model is sufficient. The **Base** branch is a width-8 NAFNet-based model and is used when stronger restoration is expected to provide additional benefit.

### 3.2 Gain-Based Router

For each training sample, we compute PSNR values for Input, Tiny, and Base: $P_I$, $P_T$, and $P_B$. We define the gain targets

$$g_{T-I} = P_T - P_I, \qquad g_{B-T} = P_B - P_T.$$

The first gain measures whether Tiny improves over bypassing restoration. The second measures whether Base provides enough additional benefit over Tiny.

FastGainRouter is implemented as a random-forest regressor using nine lightweight input statistics: grayscale mean and standard deviation, dark and bright pixel ratios, saturation mean and standard deviation, HSV value mean and standard deviation, and Laplacian variance. Given predicted gains $\hat{g}_{T-I}$ and $\hat{g}_{B-T}$, the router selects

$$a(x) = \begin{cases} \text{Input}, & \hat{g}_{T-I} < \tau_{\text{restore}}, \\ \text{Base}, & \hat{g}_{B-T} > \tau_{\text{base}}, \\ \text{Tiny}, & \text{otherwise.} \end{cases}$$

The thresholds are selected on an inner validation split.

## 4 Experiments

### 4.1 Dataset and Splits

We use the official SIDL train and validation splits. The official validation set is held out for final evaluation only. Since SIDL does not provide a separate development split for our routing policy, we construct two internal splits from the official training set: an inner training split (*train_inner*) and an inner validation split (*es_val_inner*). These names are specific to this project and are used to distinguish our internal protocol from the official SIDL split.

The *train_inner* split is the source of all trainable components in the final pipeline. It is used to train the Tiny and Base restoration branches. After the restoration branches are fixed, the same split is also used to generate router supervision: we run Input, Tiny, and Base on each train_inner sample and compute the Tiny-over-Input and Base-over-Tiny gains against the reference. Thus, train_inner defines the data from which the router learns how the trained restoration branches behave.

The *es_val_inner* split is not used for gradient updates of the restoration branches or the router. It is used for model selection and policy calibration: Tiny checkpoint selection, early stopping, and threshold selection for FastGainRouter. This split is therefore a development set inside the official training data, not an additional training set.

The official validation split is used only once the restoration branches, router, and routing thresholds have been fixed. No image from the official validation split is used for branch training, router training, early stopping, or threshold tuning. This protocol allows us to report final results on a held-out validation set while still tuning the model within the official training data.

All internal splits are constructed at the case level rather than at the patch level. SIDL contains multiple image patches that can originate from the same scene case. If patches from the same case were split across train and

**Table 1:** Split usage in the final experimental protocol. The inner splits are project-specific subdivisions of the official SIDL training split.

| Split | Source | Samples | Role in this work |
|---|---|---|---|
| train_inner | official train | 51,264 | branch training and router target generation |
| es_val_inner | official train | 5,616 | early stopping, checkpoint selection, threshold tuning |
| official val | official val | 5,520 | held-out final evaluation only |

validation subsets, the validation results could be inflated by scene-level leakage. The case-level split reduces this risk by ensuring that all patches from the same case are assigned to the same internal split.

### 4.2 Operating Points

FastGainRouter uses threshold-based routing, so different thresholds produce different quality-latency operating points. We report two representative settings selected on the inner validation split. The **Balanced** setting uses thresholds that prioritize restoration quality while still reducing average latency relative to Always Base. The **Efficient** setting uses a more aggressive policy that reduces Base usage further and allows a small PSNR drop for larger latency reduction. These names are fixed before the final official validation evaluation and correspond to two points on the same routing trade-off curve, not to separately trained restoration models.

### 4.3 Implementation and Latency Protocol

Tiny and Base are NAFNet variants with widths 4 and 8, respectively. Tiny candidates with learning rates $1 \times 10^{-3}$ and $5 \times 10^{-4}$ are compared on the inner validation split, and the $1 \times 10^{-3}$ model is used. After training Tiny and Base, we run them on train_inner to generate gain targets for the router. Router thresholds are selected on es_val_inner.

We report PSNR, SSIM, LPIPS, and latency. For dynamic routing, latency is a measured component-based pipeline estimate:

$$L = L_{\text{feature}} + L_{\text{router}} + L_{\text{branch}}.$$

This is not a fully optimized deployment runtime; it is intended to compare average computation under a batch/offline restoration setting. For fixed baselines, the latency corresponds to the measured validation pipeline latency of the selected branch under the same evaluation procedure. For FastGainRouter, the reported latency includes fast feature extraction, batch router prediction, and the inference time of the branch selected for each sample.

## 5 Results

### 5.1 Main Quantitative Comparison

The main comparison evaluates whether adaptive routing can improve the PSNR-latency trade-off over fixed-branch inference. Always Base is the strongest fixed-branch baseline in terms of PSNR, SSIM, and LPIPS among the three main branches, so it serves as the primary reference point. Always Tiny represents a lower-cost restoration baseline, while Always Input serves as a no-restoration lower bound.

Ours-Balanced achieves a near-Base operating point with lower latency. It improves PSNR over Always Base by 0.031 dB while reducing latency by 9.1%. This gain is small in absolute PSNR, but it is meaningful for the goal of this work because the stronger fixed branch is not invoked for every input.

Ours-Efficient shifts the policy toward lower computation. Compared with Always Base, it reduces latency by 18.5% with a 0.066 dB PSNR drop. Compared with Always Tiny, it improves PSNR by 0.040 dB while reducing latency by 2.1%. Thus, the efficient setting is not merely a lower-cost variant of Base; it also provides a better

**Table 2:** Main comparison on the official validation set. Latency is measured as component-based pipeline latency, not pure forward micro-benchmark latency. Relative latency reduction is computed with respect to Always Base, which serves as the strongest fixed-branch baseline.

| Method | PSNR ↑ | SSIM ↑ | LPIPS ↓ | Latency (ms/img) ↓ | ΔPSNR vs Base | Rel. Lat. Red. vs Base |
|---|---|---|---|---|---|---|
| Always Input | 21.6725 | 0.8070 | 0.2573 | 0.00 | -1.4210 | N/A |
| Always Tiny | 22.9878 | 0.8082 | 0.2420 | 99.31 | -0.1057 | 16.7% |
| Always Base | 23.0935 | 0.8145 | 0.2399 | 119.29 | 0.0000 | 0.0% |
| Ours-Balanced | 23.1250 | 0.8141 | 0.2406 | 108.43 | **+0.0315** | **9.1%** |
| Ours-Efficient | 23.0276 | 0.8119 | 0.2424 | 97.19 | **-0.0659** | **18.5%** |

PSNR-latency point than always using Tiny.

The improvement should be interpreted specifically as a PSNR-latency trade-off. Ours-Balanced does not uniformly dominate Always Base across SSIM and LPIPS, and Ours-Efficient slightly worsens LPIPS compared with Always Tiny. For this reason, we use PSNR-latency as the main quantitative claim and discuss the remaining metric trade-offs later.

## 5.2 Operating Points and Branch Usage

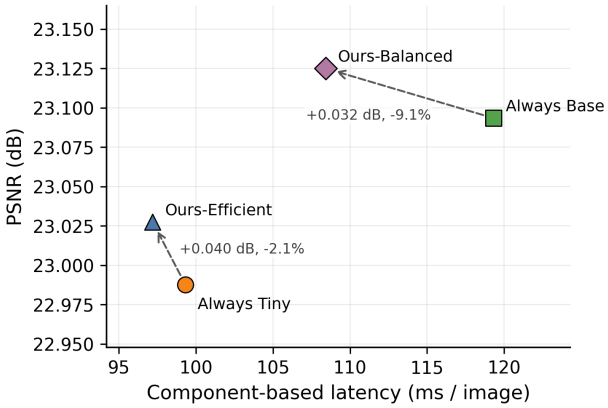
The two FastGainRouter settings correspond to different calibrated operating points of the same gain-based router. In the balanced setting, $(\tau_{\text{restore}}, \tau_{\text{base}}) = (0.3, 0.1)$, which makes the policy more willing to apply restoration and to invoke Base. This setting selects Base for 49.3% of validation samples and therefore preserves near-Base restoration quality while reducing average latency.

The efficient setting uses larger thresholds, $(\tau_{\text{restore}}, \tau_{\text{base}}) = (0.6, 0.5)$. This makes the router more conservative about restoration and more selective about Base selection. As a result, Base usage decreases from 49.3% to 13.2%, while Tiny usage increases from 39.6% to 71.7%. This shift in branch usage explains why Ours-Efficient achieves a larger latency reduction with only a small PSNR loss.

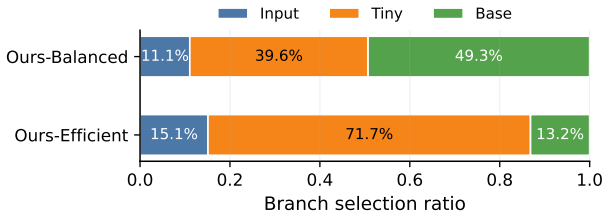
Figure 2 visualizes the resulting PSNR-latency behavior and branch usage. Ours-Balanced moves from Always Base toward lower latency while slightly improving PSNR, whereas Ours-Efficient moves from Always Tiny toward higher PSNR while also reducing latency slightly. The branch-usage plot shows that the efficient setting mainly reduces computation by replacing many Base decisions with Tiny decisions, rather than by simply bypassing restoration. Since the PSNR axis is zoomed, the visual trends should be interpreted together with the numerical values in Table 2.

**Table 3:** Operating points used by FastGainRouter. Thresholds are selected on the inner validation split before final evaluation. Branch usage is reported on the official validation set. The selection criterion describes the internal threshold-search constraint used to choose each operating point.

| Setting | $\tau_{\text{restore}}$ | $\tau_{\text{base}}$ | Input / Tiny / Base (%) | Selection criterion |
|---|---|---|---|---|
| Balanced | 0.3 | 0.1 | 11.1 / 39.6 / 49.3 | PSNR $\geq$ Always Base $-0.1$ dB |
| Efficient | 0.6 | 0.5 | 15.1 / 71.7 / 13.2 | PSNR $\geq$ Always Tiny $-0.03$ dB |




**(a)** PSNR-latency trade-off.




**(b)** Branch selection ratio.

**Figure 2:** FastGainRouter provides PSNR-latency operating points between Always Tiny and Always Base. The PSNR axis in (a) is zoomed to show small differences among restoration methods. Dashed arrows indicate improvements in the PSNR-latency plane.

## 5.3 Branch Diversity and Gain Analysis

Adaptive routing is useful only if different inputs prefer different restoration budgets. The branch-level statistics in Table 4 support this condition. Under PSNR, Input, Tiny, and Base are optimal for 27.0%, 27.8%, and 45.2% of validation samples, respectively. This indicates that no single branch is optimal for all inputs, and that fixed-model restoration can be inefficient or unnecessary for a non-trivial subset of samples.

The gain statistics show why selective use of Base is reasonable. Tiny improves over Input by 1.315 dB on average, and 70.3% of samples gain more than 0.2 dB from Tiny restoration. In contrast, Base improves over Tiny by only 0.106 dB on average, with a median gain of 0.061 dB. Thus, Tiny provides most of the restoration benefit for many inputs, while Base is useful mainly for the subset where the additional gain justifies its cost.

**Table 4:** Branch diversity and gain analysis on the official validation set.

**(a)** Optimal branch ratio.

| Metric | Input | Tiny | Base |
|---|---|---|---|
| PSNR | 27.0% | 27.8% | 45.2% |
| LPIPS | 24.7% | 28.0% | 47.3% |

**(b)** Gain statistics.

| Gain | Mean | Median | >0.2 |
|---|---|---|---|
| Tiny − Input | 1.315 | 1.776 | 70.3% |
| Base − Tiny | 0.106 | 0.061 | 41.7% |

## 5.4 Qualitative Examples

The qualitative examples provide a visual check of the three routing behaviors observed in the quantitative analysis. The first row corresponds to a bypass case, where the router selects Input and avoids unnecessary restoration. The second row shows a Tiny-sufficient case, where the lightweight branch produces a restoration result comparable to the stronger branch. The third row shows a Base-selected case, where the router assigns the input to the stronger restoration branch.



**Figure 3:** Qualitative examples of adaptive branch selection. From left to right in each row: Input, Tiny, Base, and GT. The red outline marks the branch selected by FastGainRouter using only the degraded input.

These examples are not intended to prove visual superiority for all samples. Rather, they illustrate why a single fixed branch can be inappropriate: some inputs should be left unchanged, some can be handled by Tiny, and some still benefit from Base. The selected branch is determined only from the degraded input, while the ground truth is shown only for visual comparison.

## 6 Discussion and Conclusion

FastGainRouter demonstrates that adaptive restoration budget selection can be useful on SIDL, but the results should be interpreted within the scope of this study. The method is designed as a feasibility study for input-adaptive computation allocation, not as a fully optimized deployment system or a replacement for restoration backbone research. Its main value is to show that dirty-lens restoration can be studied not only as a fixed-model restoration problem, but also as a routing problem that decides how much computation each input requires.

**Metric trade-offs.** The balanced mode slightly improves PSNR over Always Base, while SSIM and LPIPS remain comparable rather than uniformly better. The efficient mode trades a small PSNR loss for a larger latency reduction. Thus, our claim is a favorable PSNR-latency trade-off, not dominance across every quality metric. This distinction is important because PSNR measures pixel-level fidelity, whereas perceptual differences may be better reflected by LPIPS and qualitative examples.

**Latency estimate.** The reported latency is a component-based pipeline estimate. It includes feature extraction, batch router prediction, and selected branch inference, but it is not a fully optimized deployment runtime. A deployment-oriented implementation should optimize the router or replace it with a lightweight neural module suitable for low-overhead single-image inference.

**Router target generation.** The router is trained using gain targets generated from train_inner, which is also used to train Tiny and Base. This does not affect the official validation set, which is held out for final evaluation, but it is not the strictest possible protocol. A stricter design could use cross-fitting or a separate router-training split so that router targets are generated on restoration-unseen cases.

**Branch scope.** This work uses a simple Input/Tiny/Base branch family. Stronger backbones or more diverse branches could change the routing behavior and are left for future work. Overall, the observed branch diversity and PSNR-latency trade-offs suggest that adaptive budget selection complements fixed-model SIDL restoration.

## References


[1] Han Cai, Chuang Gan, Tianzhe Wang, Zhekai Zhang, and Song Han. Once-for-all: Train one network and specialize it for efficient deployment. In *International Conference on Learning Representations*, 2020.

[2] Liangyu Chen, Xiaojie Chu, Xiangyu Zhang, and Jian Sun. Simple baselines for image restoration. In *Computer Vision – ECCV 2022*, volume 13667 of *Lecture Notes in Computer Science*, pages 17–33, Cham, 2022. Springer.

[3] Sooyoung Choi, Sungyong Park, and Heewon Kim. SIDL: A real-world dataset for restoring smartphone images with dirty lenses. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 2545–2554, 2025.

[4] Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. BranchyNet: Fast inference via early exiting from deep neural networks. In *2016 23rd International Conference on Pattern Recognition (ICPR)*, pages 2464–2469, 2016.

[5] Xin Wang, Fisher Yu, Zi-Yi Dou, Trevor Darrell, and Joseph E. Gonzalez. SkipNet: Learning dynamic routing in convolutional networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 409–424, September 2018.

[6] Jiahui Yu, Linjie Yang, Ning Xu, Jianchao Yang, and Thomas Huang. Slimmable neural networks. In *International Conference on Learning Representations*, 2019.

[7] Syed Waqas Zamir, Aditya Arora, Salman Khan, Munawar Hayat, Fahad Shahbaz Khan, and Ming-Hsuan Yang. Restormer: Efficient transformer for high-resolution image restoration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5728–5739, June 2022.

[8] Syed Waqas Zamir, Aditya Arora, Salman Khan, Munawar Hayat, Fahad Shahbaz Khan, Ming-Hsuan Yang, and Ling Shao. Multi-stage progressive image restoration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14821–14831, June 2021.